

Java-Script_WEEK2-Day - 2 Hands On - Gurunath Rageni

PROBLEM 1: Student Marks Analyzer (LEVEL-1)

Scenario:

You are developing a small utility for a teacher to analyze student marks stored in an array.

Requirements

- Store student marks in an array.
- Calculate total and average marks using array methods.
- Display pass/fail result based on average.
- Use let/const appropriately.
- Use arrow functions for calculations.
- Display output using template literals.

Technical Constraints

- Must use array methods like reduce() and map().
- Use only ES6+ syntax.
- No external libraries.
- All logic must be inside modular JavaScript file.

Code:

```
<!DOCTYPE html>

<html lang="en">

<head><title>Arrays</title></head>

<body>

  <h2>Student Marks Analysis</h2>

  <div id="output"></div>

  <script>

    const studentMarks = [80, 92, 68, 99, 85];

    const calculateTotal = (marks) =>

      marks.reduce((total, mark) => total + mark, 0);
```

```
const calculateAverage = (marks) =>
  calculateTotal(marks) / marks.length;
const getResult = (average) =>
  average >= 40 ? "Pass " : "Fail ";
const analyzeMarks = (marks) => {
  const total = calculateTotal(marks);
  const average = calculateAverage(marks);
  const result = getResult(average);
  // Print inside browser page using template literals.
  document.getElementById("output").innerHTML = `
    <p><strong>Marks:</strong> ${marks.join(", ")}</p>
    <p><strong>Total:</strong> ${total}</p>
    <p><strong>Average:</strong> ${average.toFixed(2)}</p>
    <p><strong>Result:</strong> ${result}</p>`;
};
analyzeMarks(studentMarks);
</script>
</body>
</html>
```

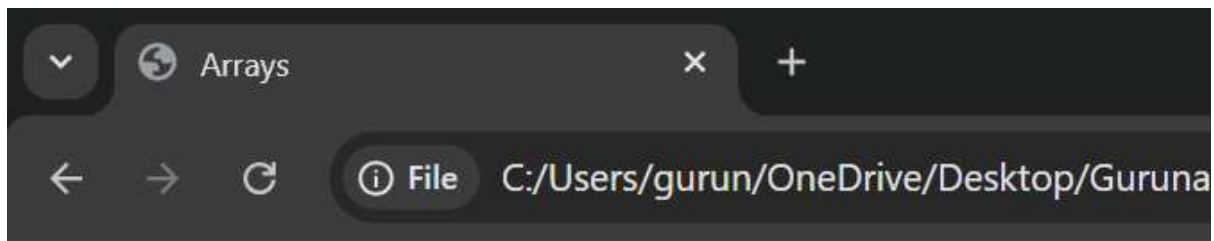
Code screenshot:

<> index.html M X

WEEK_2 > Day2_handson_is > Problem1 > <> index.html > html > body > script > analyzeMarks

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head><title>Arrays</title></head>
4  <body>
5      <h2>Student Marks Analysis</h2>
6      <div id="output"></div>
7      <script>
8          const studentMarks = [80, 92, 68, 99, 85];
9          const calculateTotal = (marks) =>
10             marks.reduce((total, mark) => total + mark, 0);
11          const calculateAverage = (marks) =>
12             calculateTotal(marks) / marks.length;
13          const getResult = (average) =>
14             average >= 40 ? "Pass " : "Fail ";
15          const analyzeMarks = (marks) => {
16             const total = calculateTotal(marks);
17             const average = calculateAverage(marks);
18             const result = getResult(average);
19             // Print inside browser page using template literals.
20             document.getElementById("output").innerHTML = `
21                 <p><strong>Marks:</strong> ${marks.join(", ")}</p>
22                 <p><strong>Total:</strong> ${total}</p>
23                 <p><strong>Average:</strong> ${average.toFixed(2)}</p>
24                 <p><strong>Result:</strong> ${result}</p>`;
25             };
26          analyzeMarks(studentMarks);
27      </script>
28  </body>
29  </html>
```

Output Screenshot:



Student Marks Analysis

Marks: 80, 92, 68, 99, 85

Total: 424

Average: 84.80

Result: Pass

Explanation:

This code uses ES6 arrow functions and the `reduce()` method to calculate total and average marks in a modular way.

It applies a ternary operator to determine pass or fail based on the average.

Finally, it updates the webpage dynamically using template literals and DOM manipulation with `innerHTML`.

PROBLEM 2: Product Cart Summary (LEVEL-1)

Scenario:

Build a simple shopping cart summary system.

📌 Requirements

- Store product objects (name, price, quantity) in an array.
- Calculate total cart value.

- Display formatted invoice using template literals.
- Use arrow functions.
- Export calculation function from a module.

Technical Constraints

- Use map() and reduce().
- Use ES6 modules (export/import).
- No DOM manipulation required.

Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>cart</title>

</head>

<body>

<script type="module">

  import { generateInvoice } from "./cart.js";

  const cart = [

    { name: "Laptop", price: 50000, quantity: 1 },

    { name: "Headphones", price: 2000, quantity: 2 },

    { name: "Mouse", price: 500, quantity: 3 }

  ];

  console.log(generateInvoice(cart));

</script>

</body>

</html>

// cartUtils.js

export const calculateTotal = (products) =>

  products.reduce(
```

```
(total, product) => total + product.price * product.quantity,0);
```

```
export const generateInvoice = (products) => {
```

```
  const total = calculateTotal(products);
```

```
  const items = products
```

```
    .map(product => `
```

```
      ${product.name}
```

```
      Price: ₹${product.price}
```

```
      Quantity: ${product.quantity}
```

```
      Subtotal: ₹${product.price * product.quantity}
```

```
      -----` ).join("");
```

```
  return `
```

```
    SHOPPING CART INVOICE
```

```
    =====
```

```
    ${items}
```

```
    TOTAL AMOUNT: ₹${total}
```

```
    =====
```

```
  `;
```

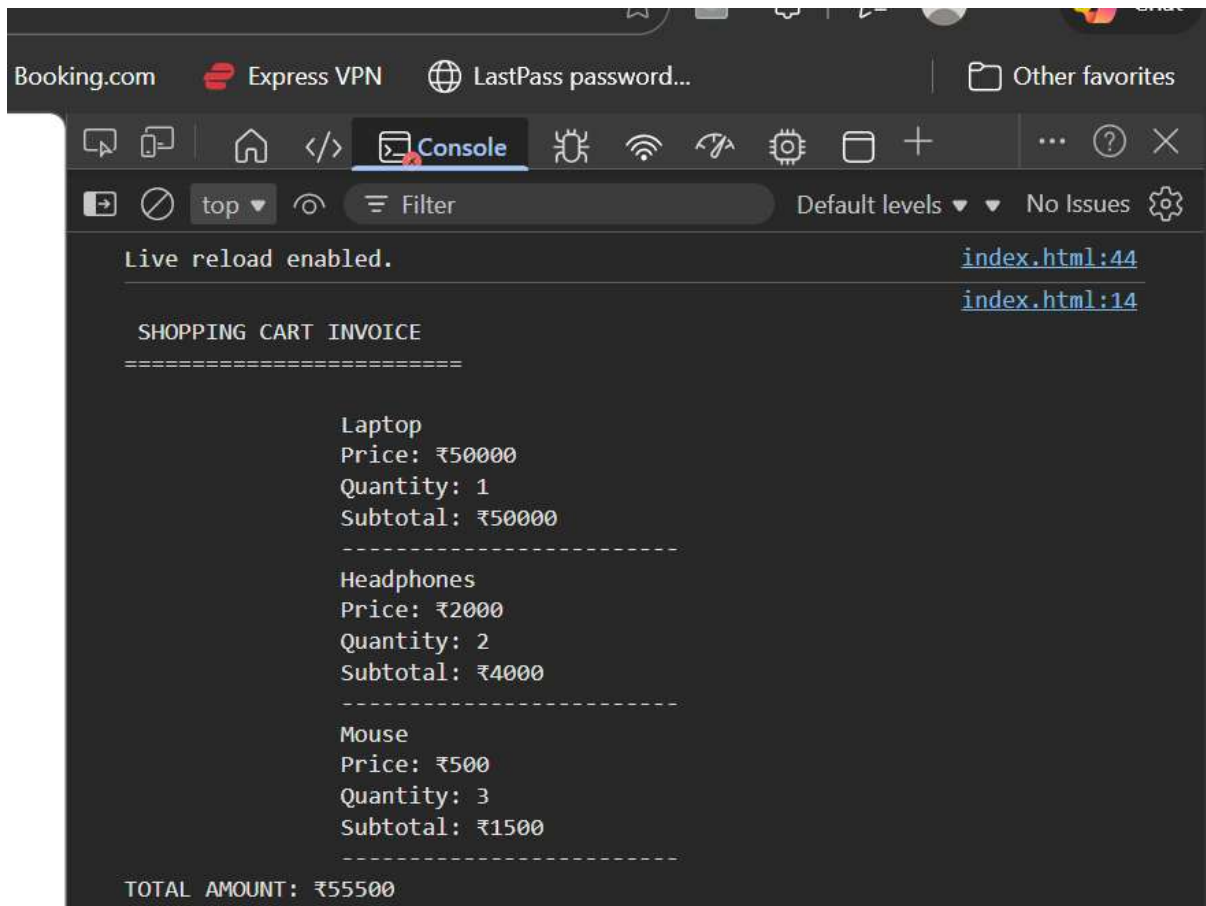
```
};
```

Code Screenshots:

```
<> index.html ...\Problem1 M    <> index.html ...\Problem2 X
WEEK_2 > Day2_handson_is > Problem2 > <> index.html > { } "index.html" > html > body > script
1  <!DOCTYPE html>
2  <html>
3  <head>
4  |   <title>cart</title>
5  </head>
6  <body>
7  <script type="module">
8  |   import { generateInvoice } from "../cart.js";
9  |   const cart = [
10 |     { name: "Laptop", price: 50000, quantity: 1 },
11 |     { name: "Headphones", price: 2000, quantity: 2 },
12 |     { name: "Mouse", price: 500, quantity: 3 }
13 |   ];
14 |   console.log(generateInvoice(cart));
15 </script>
16 </body>
17 </html>
```

```
<> index.html M    JS cart.js    X
WEEK_2 > Day2_handson_is > Problem2 > JS cart.js > calculateTotal
1  // cartUtils.js
2
3  export const calculateTotal = (products) =>
4  |   products.reduce(
5  |     (total, product) => total + product.price * product.quantity, 0);
6
7  export const generateInvoice = (products) => {
8  |   const total = calculateTotal(products);
9  |
10 |   const items = products
11 |     .map(product => `
12 |       ${product.name}
13 |       Price: ₹${product.price}
14 |       Quantity: ${product.quantity}
15 |       Subtotal: ₹${product.price * product.quantity}
16 |       -----`)
17 |     .join("");
18 |   return `
19 |   SHOPPING CART INVOICE
20 |   =====
21 |   ${items}
22 |   TOTAL AMOUNT: ₹${total}
23 |   =====
24 | `;
25 | };
```

Output Screenshot:

A screenshot of a web browser's developer console. The browser's address bar shows 'Booking.com', 'Express VPN', and 'LastPass password...'. The console tab is active, showing 'Live reload enabled.' at the top. Below it, a message 'SHOPPING CART INVOICE' is displayed, followed by a detailed list of items: 'Laptop' (Price: ₹50000, Quantity: 1, Subtotal: ₹50000), 'Headphones' (Price: ₹2000, Quantity: 2, Subtotal: ₹4000), and 'Mouse' (Price: ₹500, Quantity: 3, Subtotal: ₹1500). The total amount is ₹55500. The console also shows links to 'index.html:44' and 'index.html:14'.

Explanation:

This code demonstrates ES6 module usage by exporting reusable functions (`calculateTotal` and `generateInvoice`) and importing them using `type="module"`.

The `reduce()` method calculates the total cart value by multiplying price and quantity, while `map()` formats each product's invoice details dynamically.

It follows modular design principles, improving code reusability, separation of concerns, and maintainability in JavaScript applications.

PROBLEM 3: Weather Data Fetcher (LEVEL-2)

Scenario:

Create an application that fetches weather data asynchronously.

Requirements

- Fetch data from a public API using `fetch()`.

- Implement version using Promises.
- Implement version using async/await.
- Handle errors properly.
- Display formatted weather report.

Technical Constraints

- Use async/await.
- Use template literals.
- Use arrow functions.
- Must include proper error handling using try/catch.

Code:

```
<input type="text" id="city" placeholder="Enter city">
<button onclick="fetchWeather()">Get Weather</button>
<div id="result"></div>
```

```
<script>
const apiKey = "YOUR_API_KEY";
const fetchWeather = async () => {
  const city = document.getElementById("city").value;
  const resultDiv = document.getElementById("result");
  try {
    const response = await fetch(
      `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=metri
c`
    );
    if (!response.ok) {
      throw new Error("City not found!");
    }
    const data = await response.json();
    resultDiv.innerHTML = `
```

```

<h3>${data.name}</h3>

<p> 🌡 Temp: ${data.main.temp}°C</p>

<p> 🌤 Weather: ${data.weather[0].description}</p>

<p> 🌬 Wind: ${data.wind.speed} m/s</p>

`;
} catch (error) {
    resultDiv.innerHTML = `

${error.message}</p>`;
}
};
</script>


```

Code screenshot:

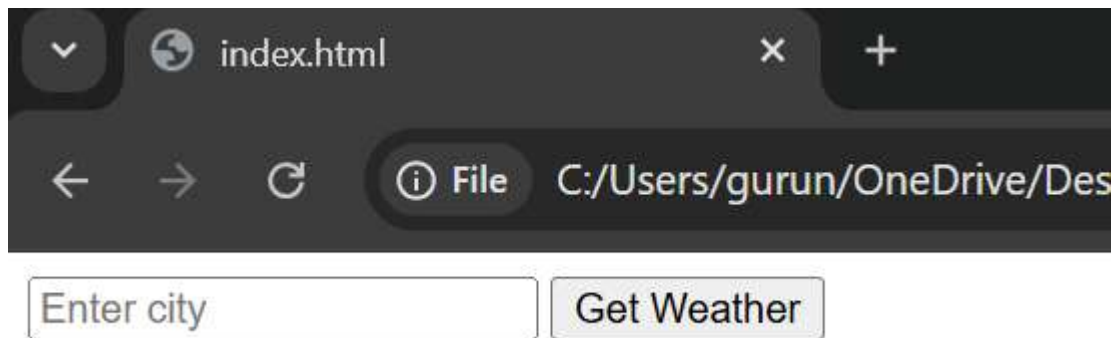
```

index.html ...\Problem1 M  index.html ...\Problem3 M ●
WEEK_2 > Day2_handson_is > Problem3 > index.html > script
1  <input type="text" id="city" placeholder="Enter city">
2  <button onclick="fetchWeather()">Get Weather</button>
3  <div id="result"></div>
4
5  <script>
6  const apiKey = "YOUR_API_KEY";
7  const fetchWeather = async () => {
8      const city = document.getElementById("city").value;
9      const resultDiv = document.getElementById("result");
10     try {
11         const response = await fetch(
12             `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=metric`
13         );
14         if (!response.ok) {
15             throw new Error("City not found!");
16         }
17         const data = await response.json();
18         resultDiv.innerHTML = `
19             <h3>${data.name}</h3>
20             <p> 🌡 Temp: ${data.main.temp}°C</p>
21             <p> 🌤 Weather: ${data.weather[0].description}</p>
22             <p> 🌬 Wind: ${data.wind.speed} m/s</p>
23         `;
24     } catch (error) {
25         resultDiv.innerHTML = `

${error.message}</p>`;
26     }
27 };
28 </script>


```

Code output:



Explanation:

This code uses an async function with `fetch()` to retrieve real-time weather data from the OpenWeather API based on user input.

It handles errors using `try...catch` and checks `response.ok` to manage invalid city names gracefully.

The DOM is updated dynamically using template literals and `innerHTML` to display temperature, weather condition, and wind speed.

PROBLEM 4: Task Manager with Async Storage (LEVEL-2)

Scenario:

Develop a task manager where tasks are saved and retrieved asynchronously.

Requirements

- Store tasks in an array.
- Create `addTask()`, `deleteTask()`, and `listTasks()` functions.
- Simulate async storage using `setTimeout` with callbacks.
- Convert callback-based logic into Promises.
- Refactor into `async/await` version.
- Use ES6 modules.

Technical Constraints

- Must demonstrate callback → promise → async/await evolution.
- Use let/const properly.
- Use arrow functions.
- Use template literals for display.

Code:

<!--PROBLEM 4: Task Manager with Async Storage (LEVEL-2)

Scenario:

Develop a task manager where tasks are saved and retrieved asynchronously.

Requirements

- Store tasks in an array.
- Create addTask(), deleteTask(), and listTasks() functions.
- Simulate async storage using setTimeout with callbacks.
- Convert callback-based logic into Promises.
- Refactor into async/await version.
- Use ES6 modules.

Technical Constraints

- Must demonstrate callback → promise → async/await evolution.
- Use let/const properly.
- Use arrow functions.
- Use template literals for display.

Learning Outcome

- Deep understanding of async JavaScript.
- Learn refactoring async code.
- Master modern ES6+ features.

!-->

```

<!DOCTYPE html>

<html>

<head>

  <title>Async Task Manager</title>

</head>

<body>

  <h2>Open Console to See Output</h2>

  <script type="module" src="app.js"></script>

</body>

</html>

```

```

import {
  addTaskCallback,
  deleteTaskCallback,
  listTasksCallback,
  addTaskPromise,
  deleteTaskPromise,
  listTasksPromise,
  addTaskAsync,
  deleteTaskAsync,
  listTasksAsync
} from "./taskmanager.js";

```

```

/* =====
1 CALLBACK USAGE
===== */

```

```

addTaskCallback("Learn JS", (msg) => {
  console.log(msg);

```

```

listTasksCallback((tasks) => {
    console.log(`Tasks: ${tasks.join(", ")}`);
});
});

/* =====

2 PROMISE USAGE
===== */

addTaskPromise("Practice DSA")
    .then(msg => {
        console.log(msg);
        return listTasksPromise();
    })
    .then(tasks => {
        console.log(`Tasks: ${tasks.join(", ")}`);
    });

/* =====

3 ASYNC / AWAIT USAGE
===== */

const runAsyncVersion = async () => {
    const msg1 = await addTaskAsync("Build Projects");
    console.log(msg1);

    const msg2 = await deleteTaskAsync("Learn JS");
    console.log(msg2);
    const tasks = await listTasksAsync();
    console.log(`Final Tasks: ${tasks.join(", ")}`);
};

runAsyncVersion();

// taskManager.js

```

```

// Simulated database

let tasks = [];

/* =====

1 CALLBACK VERSION
===== */

// Add Task (Callback)
export const addTaskCallback = (task, callback) => {
  setTimeout(() => {
    tasks.push(task);
    callback(`Task "${task}" added.`);
  }, 1000);
};

// Delete Task (Callback)
export const deleteTaskCallback = (task, callback) => {
  setTimeout(() => {
    tasks = tasks.filter(t => t !== task);
    callback(`Task "${task}" deleted.`);
  }, 1000);
};

// List Tasks (Callback)
export const listTasksCallback = (callback) => {
  setTimeout(() => {
    callback(tasks);
  }, 1000);
};

```

```
/* =====
```

2 PROMISE VERSION

```
===== */
```

```
export const addTaskPromise = (task) => {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      tasks.push(task);  
      resolve(`Task "${task}" added.`);  
    }, 1000);  
  });  
};
```

```
export const deleteTaskPromise = (task) => {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      tasks = tasks.filter(t => t !== task);  
      resolve(`Task "${task}" deleted.`);  
    }, 1000);  
  });  
};
```

```
export const listTasksPromise = () => {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(tasks);  
    }, 1000);  
  });  
};
```

```
/* =====
```

3 ASYNC / AWAIT VERSION

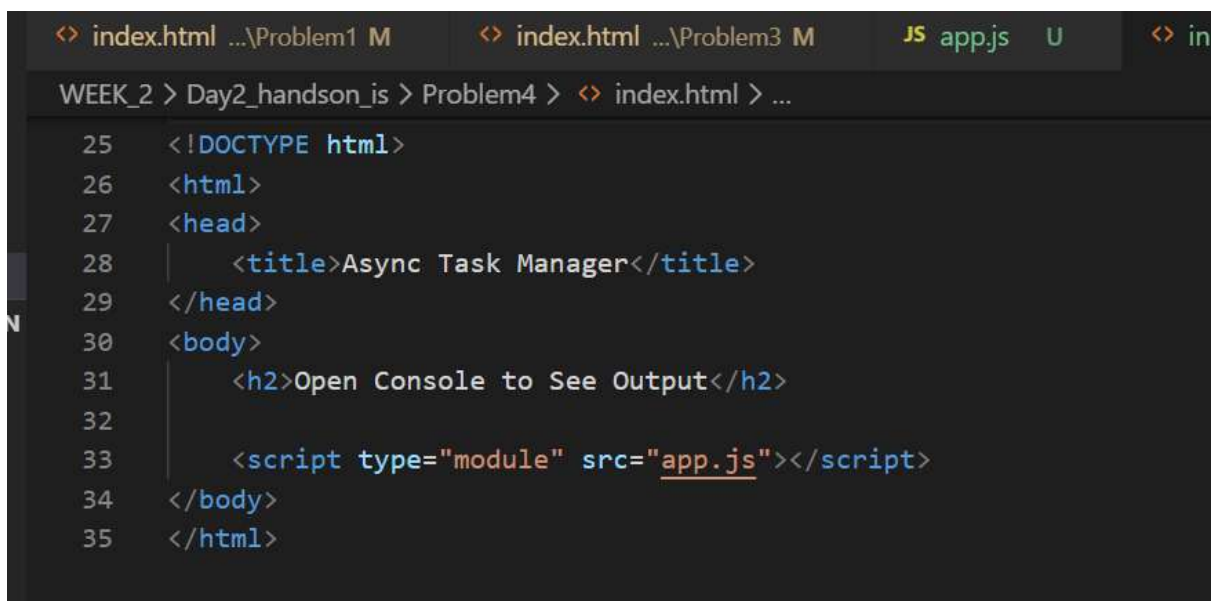
```
===== */
```

```
export const addTaskAsync = async (task) => {  
  const message = await addTaskPromise(task);  
  return message;  
};
```

```
export const deleteTaskAsync = async (task) => {  
  const message = await deleteTaskPromise(task);  
  return message;  
};
```

```
export const listTasksAsync = async () => {  
  const allTasks = await listTasksPromise();  
  return allTasks;  
};
```

Code Screenshot:



The screenshot shows a web browser window with a dark theme. The address bar displays the path: WEEK_2 > Day2_handson_is > Problem4 > index.html > ... The browser tabs show 'index.html ...\Problem1 M', 'index.html ...\Problem3 M', 'JS app.js U', and 'index.html ...\Problem4 M'. The main content area shows the HTML source code of the page, with line numbers 25 through 35. The code is as follows:

```
25 <!DOCTYPE html>  
26 <html>  
27 <head>  
28   <title>Async Task Manager</title>  
29 </head>  
30 <body>  
31   <h2>Open Console to See Output</h2>  
32  
33   <script type="module" src="app.js"></script>  
34 </body>  
35 </html>
```

WEEK_2 > Day2_handson_is > Problem4 > JS app.js > ...

```
30     .then(msg => {
31         console.log(msg);
32         return listTasksPromise();
33     })
34     .then(tasks => {
35         console.log(`Tasks: ${tasks.join(", ")}`);
36     });
37
38     /* =====
39     3 ASYNC / AWAIT USAGE
40     ===== */
41     const runAsyncVersion = async () => {
42         const msg1 = await addTaskAsync("Build Projects");
43         console.log(msg1);
44
45         const msg2 = await deleteTaskAsync("Learn JS");
46         console.log(msg2);
47         const tasks = await listTasksAsync();
48         console.log(`Final Tasks: ${tasks.join(", ")}`);
49     };
50     runAsyncVersion();
```

```
> index.html ...\Problem1 M    <> index.html ...\Problem3 M    JS app.js U X    <> index.html
WEEK_2 > Day2_handson_is > Problem4 > JS app.js > ...
1  import {
2      addTaskCallback,
3      deleteTaskCallback,
4      listTasksCallback,
5      addTaskPromise,
6      deleteTaskPromise,
7      listTasksPromise,
8      addTaskAsync,
9      deleteTaskAsync,
10     listTasksAsync
11 } from "../taskmanager.js";
12
13
14  /* =====
15  | 1 CALLBACK USAGE
16  ===== */
17
18  addTaskCallback("Learn JS", (msg) => {
19      console.log(msg);
20
21      listTasksCallback((tasks) => {
22          console.log(`Tasks: ${tasks.join(", ")}`);
23      });
24  });
25
26  /* =====
27  | 2 PROMISE USAGE
28  ===== */
29  addTaskPromise("Practice DSA")
```

Output Screenshot:

```
Task "Learn JS" added.                                app.js:19
Task "Practice DSA" added.                             app.js:31
Task "Build Projects" added.                           app.js:43
Tasks: Learn JS, Practice DSA, Build Projects          app.js:22
Tasks: Learn JS, Practice DSA, Build Projects          app.js:35
Task "Learn JS" deleted.                               app.js:46
Final Tasks: Practice DSA, Build Projects              app.js:48
[NEW] Explain console errors by using Copilot in Edge:
click  to explain an error. Learn more
>
```

Explanation:

This project demonstrates the evolution of asynchronous JavaScript by implementing a Task Manager using callbacks, Promises, and `async/await` with ES6 modules. It simulates asynchronous storage using `setTimeout`, showing how callback-based logic can lead to nested structures, which are then improved using Promise chaining for better readability.

Finally, the `async/await` version refactors the Promise-based implementation into a cleaner, synchronous-like flow while maintaining non-blocking behavior. The code follows modern ES6+ practices including `let/const`, arrow functions, template literals, and modular structure for better maintainability and scalability.